

Live Demo Script

30 to 40 minute presentation, second by second

Mohammad Kamrul Hasan · AI-Augmented QA Architect · April 2026

YOUR 35-MINUTE PLAN

- **0 to 4 min:** Open + architecture overview
- **4 to 12 min:** Demo 1 — Requirement Management LIVE
- **12 to 21 min:** Demo 2 — API Functional Test LIVE
- **21 to 25 min:** Demo 3 — Memory in action
- **25 to 29 min:** Engineering decisions (4 trade-offs)
- **29 to 32 min:** Value + roadmap + the ask
- **32 to 40 min:** Q and A

PRE-FLIGHT (do this 30 minutes before audience arrives)

- Open terminal at E:/Projects/NAPCO-Nucleus. Activate venv if needed.
- Open DB Browser for SQLite. Load nucleus_memory.db.
- Open browser tabs: GitHub repo (Actions tab), GitLab project (titucs/mvp-access), Drive folder, your inbox.
- Open NN-Architecture.pdf on screen for the overview.
- Send yourself a fake requirement email from a different address (e.g., titucse@gmail.com) to khasan@ael-bd.com. Subject: 'Add a Forgot Password link to the login page'. Body: '~2 hours of work, no priority. Send by end of week.' This becomes the demo trigger.
- Run `py -3 agent.py --task requirement-management --dry-run ONCE` silently to warm imports + verify auth.
- Close anything visually noisy. Cell phone on silent. Notifications muted.

SAY (verbatim, slowly, calmly): "You have all heard about AI agents this past year. The Anthropic and OpenAI announcements, the demos, the LinkedIn posts. Tonight you are going to see one running on our infrastructure, on our data, doing real work for our team. Not a slide deck about what AI might do. A working agent that filed thirteen real client requirements into our GitLab backlog this morning, from emails and meeting recordings, while we slept. I am Mohammad. I am a QA architect by trade. I designed and shipped this end to end by directing AI through Claude Code. Let me show you what that looks like."

SHOW (3 min): Switch to NN-Architecture.pdf page 1. Walk through the metrics row (9 workflows, 31 tools, 4 tests at 2am, 2 daily emails, \$200/month). Then point at the 'How one run works' card. Spend 30 seconds on each step.

TRANSITION: 'OK enough talking about it. Let me show you the agent actually doing something.'

DEMO 1 — REQUIREMENT MANAGEMENT (LIVE)

4 - 12 MIN

SAY: 'I just sent myself an email with a fake client requirement. Watch the agent pick it up.'

TYPE in terminal:

```
py -3 agent.py --task requirement-management --dry-run
```

NARRATE WHILE IT RUNS (~3 min): 'See those log lines? It is connecting to our Gmail mailbox via IMAP... now it is checking the Drive folder for new recordings... now Claude is reading the inbox files...' Pause. 'Now Claude is splitting each email into 3-hour tasks. Watch the JSON come back.' Pause again. 'And now it is checking each task against memory to see if we have filed it before. Idempotent dedup. Re-running this never duplicates work.'

EXPECT TO SEE:

- Logs from requirements_inbox connecting to imap.gmail.com
- 'Found N ingestable file(s)' from drive_ingester
- Claude narrating its splitting decisions
- 'created: X, skipped: Y, failed: 0' summary at the end

SHOW (after run finishes): Switch to GitLab tab. Refresh. Point at the new issue or the most recent one. 'Here is the actual issue, in our actual project. The body has the source file reference. The estimate is in the body. The labels are applied.'

FALLBACK if the run fails:

- 'OK this just failed because X. Watch how the agent logs the error to memory and exits cleanly. That is by design. Failures are visible, not silent.' Then open DB Browser, show the activity_logs row with result='error:...'
- Backup demo: 'Let me show you what an earlier run produced.' Refresh GitLab and point at the 13 issues from this morning's automatic run.

SAY: *'Now let me show you the test side. This is the agent running our Newman / Postman collection against the live API. Notice that I am about to run it just like the scheduled 02:00 BDT cron does, no special handling.'*

TYPE in terminal:

```
py -3 agent.py --task api-functional-test
```

NARRATE WHILE IT RUNS (~6 min): 'Pre-flight check first. The agent is pinging the login endpoint to make sure the server is alive before burning a test budget on a dead target.' Pause. 'Now Newman is firing. You will see one of these for every request that fails an assertion.' Pause. 'And here is the interesting part. Once Newman finishes, the agent is going to open known_bugs.py, match each failure against the xfail markers, and tell you which failures are real bugs versus which ones are already tracked.' Wait for results. Then: '288 of 315 passed. 27 failed. The agent is going to interpret each failure now.'

EXPECT TO SEE:

- 'API health check passed: server at ... responded in Xms'
- Newman test output streaming through
- 'API Test: PARTIAL — 288/315 passed' summary line
- Claude reasoning aloud about specific failures
- PDF path printed at the end

SHOW (after run finishes): Open the generated PDF. Point at one failure card. 'See the classification: real_bug versus known_bug. The reasoning column is Claude saying why. This is what the test team will read tomorrow morning.'

FALLBACK if the run hangs or fails:

- Ctrl-C. Then: 'OK that is the network or the API being slow. Let me show you a result from earlier today.' Open the email I auto-sent earlier showing 91.4% pass rate.
- Backup demo: open MVP-Access-API-Test/reports/Test_Report_20260426_*.pdf directly.

SAY: *'Now the secret sauce. Every run I just did wrote rows to a SQLite database. That database is committed to the git repo. It is the agent's memory across runs.'*

SHOW: Switch to DB Browser. Already has nucleus_memory.db open.

- Click the activity_logs table. Scroll to today. Point: 'Every action the agent took, with timestamp and result.'
- Click requirements_seen. 'Every requirement it has filed, with normalized titles for fuzzy dedup. This is why re-running never creates duplicates.'
- Click test_run_history. 'Every test run, with pass / fail counts and the PDF path. This is what the morning Daily Report reads from.'

SAY: *'And because the database lives in git, when the runner picks up tomorrow morning at 02:00, it has full memory of every prior run. No external database to provision. Disaster recovery is git clone.'*

FALLBACK:

- If DB Browser hangs: 'Let me query it from the terminal instead.' Run `py -3 -c "import memory; print(memory.stats())"`

ENGINEERING DECISIONS (4 TRADE-OFFS, ~1 MIN EACH)

25 - 29 MIN

SAY: *'Now I want to spend 4 minutes on the engineering decisions, because the engineers in the room will care about these.'*

1. Claude Max via local CLI, not ANTHROPIC_API_KEY. 'Cost is fixed monthly, not per token. Eight scheduled workflows fire 30 times a day. On the API that would be real money. About \$200 a month flat.'

2. Algorithms in prompts, not Python. 'I deleted 638 lines of regex-based heuristics last week and moved that work into prompts. Adding a new failure category is a markdown edit, not a code review.'

3. Memory committed to git. 'The SQLite database is in the repo. Cloning the project gives you the agent's full history. Disaster recovery is git clone.'

4. Self-hosted runner. 'Direct access to staging behind VPN, direct access to TFS, direct access to IIS UNC paths. None reachable from a hosted runner. Secrets stay on our infrastructure.'

SAY (value, ~90 seconds):

"What this delivers today: 13 client requirements automatically filed in GitLab from this morning. 27 backend bugs classified across the test suite. A 100-user load test that surfaced real staging strain we acted on. A consolidated daily report that replaces six fragmented per-suite emails. And it does this every day on a fixed monthly cost."

SAY (roadmap, ~60 seconds):

"What is next: fix the runner permission so all four test workflows can see the sibling test code. Plug TFS and IIS credentials into the CICD workflow so we own the deploy cycle too. Add Teams chat as a third requirement intake source. Add fixture self-healing for E2E so when staging changes a button text, the agent updates the selector instead of the test failing."

SAY (the ask, ~30 seconds):

"To unlock the next phase I need three operational things from this team: (1) IT to grant the runner service account read access to E:/Projects, (2) DevOps to add the six TFS and IIS secrets to GHA, (3) approval to keep the Claude Max subscription as a line item. Once those land, NAPCO Nucleus owns the full code-to-test-to-deploy cycle every night."

Q AND A SURVIVAL

32 - 40 MIN

- **Pause 2 seconds before answering.** Looks thoughtful, not slow.
- **Anchor every answer back to the thesis.** 'QA architect plus AI equals senior dev team output.'
- **For the developer-question (most likely): own the AI partnership.** See Study Guide page 4. 'I designed it. AI implemented under my direction. The system is real, the issues are in GitLab, the reports are in your inbox.'
- **If you do not know:** 'Let me check.' Open the file, look it up. Better than guessing.
- **If someone challenges hard:** 'Good question. We considered Y and chose X because of Z.' Always name the alternative.

FINAL LINE WHEN Q&A; WINDS DOWN

"Thank you. The agent is going to keep running every two hours and every morning. The next Daily Report will land in your inbox at 09:00 tomorrow. Open it and tell me what you want changed."